



School of Science and Engineering

Inspire, Impact & Transform

Adaptive Hybrid Cloud Gateway for Secure AI Chatbots

Capstone Design Final Report

Maryam Taoumi

Supervised by:

Dr. Lotfi Jawad

TABLE OF CONTENTS

ACKNOWLEDGEMENTS.....	Error! Bookmark not defined.
LIST OF FIGURES.....	4
LIST OF TABLES	5
ABSTRACT (ENGLISH)	6
RESUME (FRENCH)	7
1 INTRODUCTION.....	8
2 PROBLEM STATEMENT	10
3 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS	12
3.1 Functional Requirements	12
3.2 Non-Functional Requirements	12
4 RELATED WORK	14
4.1 Cloud-Only AI Platforms	14
4.2 On-Premises Language Models	14
4.3 Data Loss Prevention Systems	14
4.4 The Gap This Project Fills.....	14
5 SYSTEM DESIGN	15
5.1 Software Development Lifecycle.....	15
5.2 System Architecture	15
5.3 Database Design.....	17
5.4 Technology Stack and Frameworks.....	17
6 AI USE.....	18
7 IMPLEMENTATION.....	19
7.1 Multi-Rule Sensitivity Checker.....	19
7.2 ML Classifier.....	19
7.3 Router Logic.....	21
7.4 Smart Redactor	22
7.5 Local AI Handler.....	23
7.6 Cloud AI Handler	23
7.7 Chat Interface	23
8 TESTING.....	26
8.1 Unit Testing	26
8.2 Integration Testing	26
8.3 Formal Evaluation.....	27

8.4 False Positive Problem and Fix	29
9 DISCUSSION AND ANALYSIS.....	30
10 CONCLUSION	33
11 FUTURE WORK.....	34
REFERENCES	35
APPENDIX A : CODE	36
A.1 Project Source Code Structure	36
A.2 sensitivity_checker.py.....	36
A.3 router_logic.py	36
A.4 redactor.py	37
A.5 local_handler.py	37
A.6 main.py.....	37
APPENDIX B : SYSTEM DATA AND RESULTS	39
B.1 Technology Stack	39
B.2 Final Evaluation : April 19, 2026	39

LIST OF FIGURES

Figure 1 : System Architecture	16
Figure 2 : Request Processing Flowchart.....	17
Figure 3 : Multi-Rule Detection Flow	19
Figure 4 : Smart Redactor (Mixed Message Handling)	22
Figure 5 : Chat Interface (Home Page)	24
Figure 6 : Chat Interface (LOCAL Response)	24
Figure 7 : Chat Interface (CLOUD Response)	25
Figure 8 : Chat Interface (Mixed Response)	25
Figure 9 : Swagger UI interface used during integration testing	26
Figure 10 : Final evaluation summary showing accuracy, latency, and successful privacy protection.....	28
Figure 11 : Routing Distribution Chart	30
Figure 12 : MLProbability Score Before vs After Dataset Expansion	31
Figure 13 : Average Processing Latency by Route Type.....	32

LIST OF TABLES

Table 1 : Training Dataset Composition 20

Table 2 : Routing Threshold Decision Policy 21

Table 3 : Average Processing Latency by Route Type 28

Table 4 : False Positive Reduction After Dataset Expansion 29

ABSTRACT (ENGLISH)

This report presents the final implementation of the Adaptive Hybrid Cloud Gateway, a capstone project developed at Al Akhawayn University in Ifrane. The idea behind this project is simple: many organizations want to use AI chatbots to help their employees, but they are worried about their confidential data being sent to external cloud services. The solution built in this project is a gateway that sits between the user and any AI service, automatically checking each message before deciding where it should be processed. If the message contains sensitive information like salaries, emails, passwords, or ID numbers, it is handled by a local AI model running entirely inside the organization, so nothing private ever leaves the building. If the message is a safe general question, it gets forwarded to a cloud AI for a stronger and more detailed response. When a message contains both types of content, the system splits it, keeps the sensitive part local, and sends only the safe part to the cloud. The classification uses a combination of rule-based detection that checks for known sensitive patterns and a machine learning classifier trained on 258 labeled examples. The full system was tested on 22 messages and achieved 100 percent routing accuracy with zero sensitive data leakage and a confirmed privacy pass.

Keywords: hybrid cloud gateway, data privacy, sensitive data classification, on-premises AI, smart redactor

RESUME (FRENCH)

De nos jours, de nombreuses entreprises cherchent à intégrer des outils d'intelligence artificielle dans leur environnement de travail pour améliorer la productivité de leurs employés. Cependant, la grande majorité de ces outils fonctionnent entièrement en ligne, ce qui signifie que chaque message envoyé par un utilisateur quitte le réseau interne de l'organisation pour être traité sur des serveurs externes. Cette réalité pose un problème sérieux pour toute organisation qui manipule des données confidentielles, qu'il s'agisse d'informations financières, de dossiers clients, ou de données personnelles des employés. Ce projet propose une solution concrète à ce problème : une passerelle intelligente appelée Adaptive Hybrid Cloud Gateway. Cette passerelle s'intercale entre l'utilisateur et le service d'intelligence artificielle, et analyse automatiquement le contenu de chaque message avant de décider comment le traiter. Les messages contenant des informations sensibles sont traités directement en local, sans jamais quitter l'organisation. Les messages sans données confidentielles sont transmis à un service cloud qui offre des réponses de meilleure qualité. Lorsqu'un message contient à la fois des informations sensibles et une question générale, le système le découpe intelligemment et traite chaque partie de manière appropriée. Le système a été évalué sur 22 messages de test et a obtenu un taux de précision de 100 pour cent, sans aucune fuite de données sensibles vers le cloud.

Mots clés: passerelle cloud hybride, confidentialité des données, classification de sensibilité, intelligence artificielle locale, routage adaptatif

1 INTRODUCTION

Artificial intelligence chatbots are becoming genuinely useful in the workplace. Companies across all industries are starting to realize the real value they bring to their daily operations. AI tools offer significant advantages:

- They help employees get answers quickly and summarize long documents without waiting for a colleague to respond.
- They handle repetitive questions that would otherwise take time away from more important work.
- They make it possible for smaller organizations to access capabilities that previously required entire specialized teams.
- They are available around the clock, reducing delays and allowing employees to focus on higher-value tasks instead of spending time on routine information retrieval.

Despite these clear advantages, many organizations still hesitate to adopt AI chatbot tools. The reason is straightforward: most AI chatbot platforms are fully cloud-based, which means every message a user types gets sent to an external server somewhere on the internet. For a company that handles employee salaries, client records, legal documents, or financial data, that is simply not acceptable.

This project was built to solve that problem. The Adaptive Hybrid Cloud Gateway is a system that sits between the user and any AI service, automatically deciding where each message should be processed before anything gets sent anywhere. If the message contains something sensitive, it stays inside the organization and gets handled by a local AI model. If it is just a general question, it goes to a cloud AI that gives a better and more detailed answer.

And if the message is a mix of both types of content, the system is smart enough to split it and handle each part the right way.

This final report covers the complete journey of building this system across four main phases. The Research phase involved studying existing approaches to AI privacy and sensitive data detection. The Design phase focused on defining the system architecture and the routing logic. The Implementation phase covered the development of all components including the multi-rule checker, the machine learning classifier, the smart redactor, and both the local and cloud AI handlers. Finally the Testing phase evaluated the system on 22 standardized messages and confirmed 100 percent routing accuracy with zero sensitive data ever reaching the cloud.

2 PROBLEM STATEMENT

The problem is straightforward. Organizations want to use AI, but they cannot trust that their data will stay private when using cloud-based tools. When an employee types a message into a cloud AI chatbot, that message leaves the company's network entirely. There is no control over where it goes, how it is stored, or who might have access to it.

The risks this creates for companies are real and varied. A healthcare organization that uses a cloud AI chatbot risks exposing patient records to external servers, which can lead to serious regulatory violations under laws like GDPR in Europe (European Parliament, 2016). A law firm whose employees discuss case details with a cloud AI tool may unknowingly leak privileged client information to a third-party provider. A company sharing internal financial projections or merger plans through a cloud chatbot risks exposing strategic data that could reach competitors. These are not theoretical scenarios, as AI adoption grows, incidents of sensitive business data being processed and retained by external providers are becoming an increasing concern for legal, compliance, and security teams worldwide.

Several attempts have been made to address this issue, but none fully solve it. Enterprise data loss prevention tools can block sensitive content from leaving the network, but they do not offer an AI response in return, they simply block the message. Fully on-premises AI deployments protect privacy but give weaker responses and require expensive infrastructure to maintain. Some cloud AI providers offer data residency options, but the data still leaves the organization's internal systems and gets processed on external servers. None of these approaches automatically handle each message differently based on what it actually contains.

This project addresses that gap directly. The goal was not to build a new AI model, but to build the intelligent layer that decides, for every single message, where it should go

based on what it actually contains, keeping sensitive data inside the organization and letting safe content benefit from the full power of cloud AI.

3 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

3.1 Functional Requirements

FR1: The system receives user messages through a REST API and returns an AI-generated response.

FR2: Every message is classified as sensitive or safe before any AI processes it.

FR3: All sensitive patterns in a message are detected simultaneously, not just the first one found.

FR4: Fully sensitive messages are routed to a local AI model running on-premises.

FR5: Fully safe messages are routed to a cloud AI API.

FR6: Mixed messages are split into fragments, each fragment is routed independently, and the responses are merged into one reply.

FR7: Every request is logged with the routing decision, the reason, and the processing time.

FR8: The chat interface shows clear color-coded badges for LOCAL, CLOUD, and MIXED routing.

FR9: When a message is handled locally, the user sees a confirmation that no external resource was contacted.

3.2 Non-Functional Requirements

NFR1 - Privacy: Sensitive data must never reach any external server under any circumstance.

NFR2 - Accuracy: The routing system must be reliable enough that no sensitive data ever leaks to the cloud.

NFR3 - Transparency: Every routing decision must come with a human-readable explanation that can be audited.

NFR4 - Configurability: The keyword list and routing thresholds can be adjusted without touching the

code.

NFR5 - Portability: The system must run on a standard laptop without a dedicated GPU or specialized hardware.

4 RELATED WORK

4.1 Cloud-Only AI Platforms

Tools like ChatGPT, Google Gemini, and similar cloud AI assistants are powerful and easy to use, but everything goes through the cloud. Every message, every follow-up, every document snippet the user pastes in it all leaves the organization's network. Enterprise plans offer some additional controls, but the data still has to travel to an external server to be processed. For companies in finance, healthcare, or law, this is often a non-starter.

4.2 On-Premises Language Models

The release of open-source language models like LLaMA, Mistral, and Qwen has made it possible to run AI locally on company hardware. Tools like Ollama make the setup surprisingly simple. The trade-off is that these local models are smaller and less capable than the big cloud models, especially for complex reasoning tasks. They work well for many use cases but cannot match cloud AI on detailed technical or analytical questions.

4.3 Data Loss Prevention Systems

Enterprise data loss prevention tools are designed to detect and block sensitive data before it leaves an organization's network. They use pattern matching and keyword rules similar to the rule-based detection in this project. However, DLP systems operate at the network level and do not interact with AI routing logic. They cannot split a message, route each part to a different AI, and merge the responses into one reply for the user.

4.4 The Gap This Project Fills

There is no existing lightweight system that combines automatic per-message sensitivity classification, on-premises AI for sensitive content, cloud AI for safe content, smart redaction for mixed messages, and a full audit log, all in one deployable package that runs on a normal laptop. That is exactly what this project built.

5 SYSTEM DESIGN

5.1 Software Development Lifecycle

The project was built using an Agile approach, working in small iterations and testing each part before moving to the next. This turned out to be the right choice because several unexpected challenges came up during development, and being able to adjust quickly without breaking everything was important. For example, when the original local model could not run due to RAM limitations, the switch to a smaller model was made without affecting anything else in the system. The complete project folder structure is provided in [Appendix A.1](#).

5.2 System Architecture

The system has a clear layered structure. At the top is the FastAPI server which handles incoming HTTP requests and serves the chat interface. Below that is the classification and routing layer containing the multi-rule checker and the ML classifier working side by side. The smart redactor handles mixed messages by splitting them into fragments before routing. The AI processing layer has two separate handlers: one for the local model and one for the cloud API. Everything that happens gets written to the log file by the logger module.

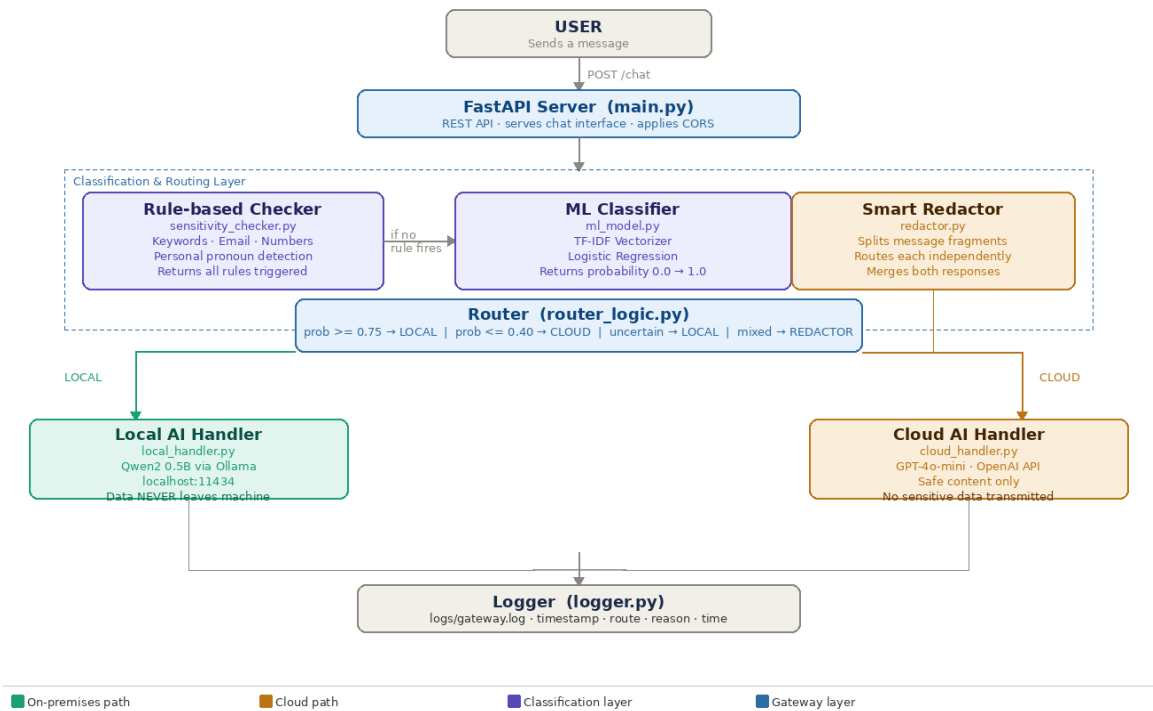


Figure 1 : System Architecture

When a message comes in, the redactor first checks if it is mixed. If it is, the sensitive fragments go to the local AI and the safe fragments go to the cloud, and the two responses are merged. If the message is not mixed, the multi-rule checker runs first. If any rule fires, the whole message goes LOCAL. If no rule fires, the ML classifier gives a probability score. A score of 0.75 or above goes LOCAL, 0.40 or below goes CLOUD, and anything in between defaults to LOCAL as a privacy-first safety measure.

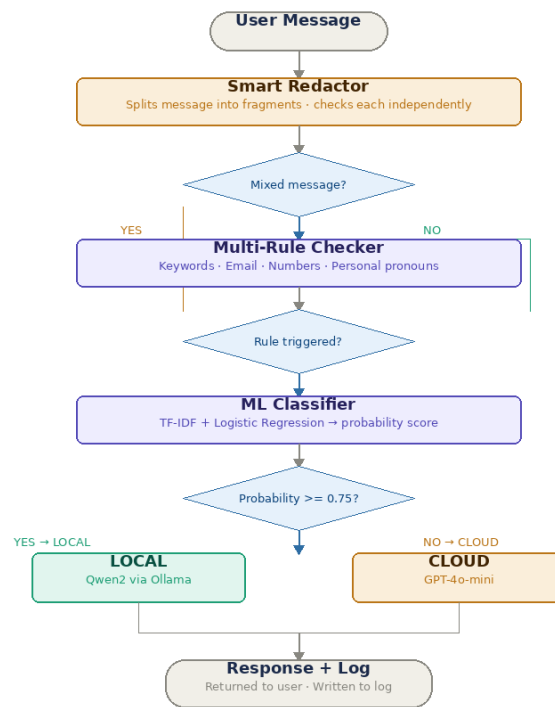


Figure 2 : Request Processing Flowchart

5.3 Database Design

The system does not use a traditional database. The trained ML model and vectorizer are saved as pickle files on disk and loaded when the server starts. All request logs are written to a plain text file called gateway.log which gets parsed by the evaluation script when needed. Keeping things simple like this meant the whole system could run on a laptop without installing any database software.

5.4 Technology Stack and Frameworks

The backend runs on Python with FastAPI and Uvicorn. The rule-based detection uses Python's built-in regular expression module. The ML classifier uses scikit-learn with TF-IDF vectorization and Logistic Regression. The local AI runs on Ollama with the Qwen2 0.5B model. The cloud AI connects to the OpenAI API with GPT-4o-mini through the official Python SDK. HTTP calls to Ollama are made using httpx. Environment variables including API keys are loaded securely using python-dotenv. The chat interface is built with React, providing a professional and responsive user experience with color-coded routing indicators, session management, and animated feedback. A complete summary of all technologies used is provided in [Appendix B.1](#).

6 AI USE

During this project, AI tools were used in a limited and specific capacity as a technical assistant for debugging and decision support. When integration errors came up during development, AI was consulted to help identify the root cause and suggest which approach would work better when there were two options to consider. For example, when the local model was returning an unexpected response format, AI helped confirm which API endpoint and response structure to use. When the system was running out of memory with the original local model, AI helped confirm that switching to a smaller model was the right decision. All architecture choices, the hybrid detection approach, the training dataset, the routing thresholds, the redactor logic, and the overall project direction were determined independently through personal research and experimentation. AI tools were used strictly as a debugging and decision support resource, equivalent to consulting documentation or asking a technical peer for a second opinion. All AI tool usage is declared here in accordance with the university academic integrity guidelines.

7 IMPLEMENTATION

7.1 Multi-Rule Sensitivity Checker

The original sensitivity checker stopped as soon as it found the first matching rule. Following supervisor feedback, this was redesigned so that every rule in the list gets checked and all triggered rules are reported together. This makes the routing decision stronger: if three independent rules all flag the same message as sensitive, there is much less risk of a wrong routing decision than if only one rule fired.

The checker now runs four independent tests on every message. It checks for sensitive keywords from a list of 20 terms including salary, confidential, bank account, credit card, password, and national ID. It checks for email address patterns using a regular expression. It checks for sequences of five or more digits which typically indicate ID numbers or account numbers. And it checks whether a personal pronoun like my or our appears together with a sensitive keyword, which helps distinguish a personal disclosure from a general question about the same topic. The full source code for the sensitivity checker is provided in [Appendix A.2](#).

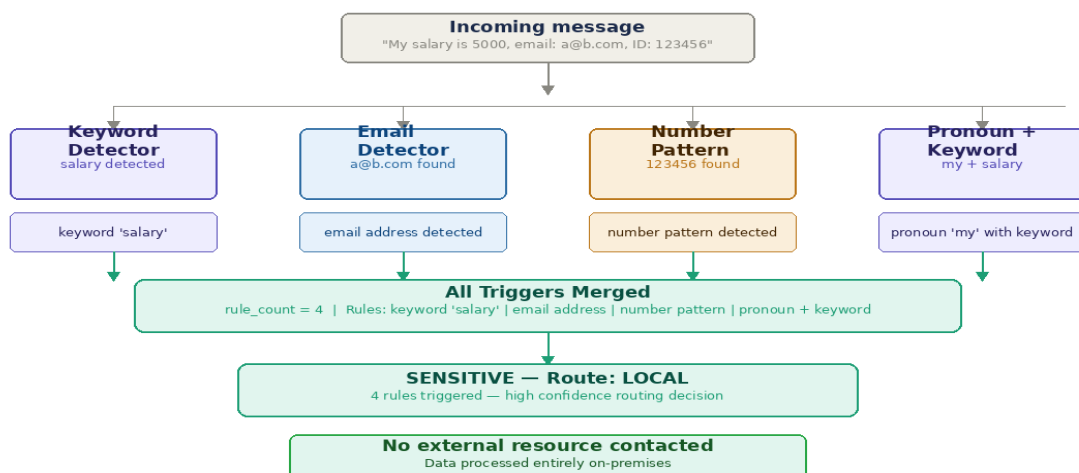


Figure 3 : Multi-Rule Detection Flow

7.2 ML Classifier

The machine learning classifier converts each message into a TF-IDF vector and uses Logistic Regression to predict a sensitivity probability between 0 and 1. The model was trained on 258 labeled examples: 50 sensitive messages covering salary figures, credentials, medical records, financial data, and confidential documents, and 208 safe messages covering general knowledge questions, personal lifestyle statements, programming topics, academic discussions, and workplace conversations. The diversity of safe examples was important because an earlier version of the model was incorrectly flagging things like Tell me a joke and I am studying computer science as potentially sensitive.

Table 1 : Training Dataset Composition

Category	Label	Example Topics	Count
Salary and payroll	Sensitive (1)	Salary figures, payroll documents, salary slips	8
Authentication credentials	Sensitive (1)	Passwords, PINs, login credentials	6
Personal identifiers	Sensitive (1)	ID numbers, SSN, passport, national ID	8
Financial data	Sensitive (1)	Bank accounts, credit cards, transfers	10
Confidential documents	Sensitive (1)	Classified reports, internal memos, private data	10
Medical records	Sensitive (1)	Patient records, medical insurance, health data	8
General AI and tech questions	Safe (0)	Machine learning, APIs, encryption, cloud computing	45
Personal lifestyle statements	Safe (0)	Hobbies, sports, cooking, travel, music	40
Academic and research topics	Safe (0)	Projects, studies, research, thesis work	35

Category	Label	Example Topics	Count
Business and workplace	Safe (0)	Meetings, teams, office, productivity	35
Programming and development	Safe (0)	Python, Git, Docker, frameworks, algorithms	53
TOTAL			258

7.3 Router Logic

The router is where the final routing decision gets made. If the rule checker found anything, the message immediately goes LOCAL without running the ML model. If nothing was found by rules, the ML probability score determines the route. A score of 0.75 or above means the model is confident the message is sensitive, so it goes LOCAL. A score of 0.40 or below means the model is confident it is safe, so it goes CLOUD. Anything between 0.40 and 0.75 is uncertain, and the system defaults to LOCAL. The reasoning is simple: it is better to occasionally give a weaker local response to a safe message than to accidentally send something sensitive to the cloud. The full source code for the router is provided in [Appendix A.3](#)

Table 2 : Routing Threshold Decision Policy

Condition	Threshold	Route	Reason
Rule-based checker triggered	Any rule fires	LOCAL	Immediate sensitive pattern detected
ML probability high	prob $\geq$ 0.75	LOCAL	High confidence sensitive message
ML probability low	prob $\leq$ 0.40	CLOUD	High confidence safe message

Condition	Threshold	Route	Reason
ML probability uncertain	$0.40 < \text{prob} < 0.75$	LOCAL	Privacy-first fallback , default to safe side
Mixed message detected	Has both sensitive and safe fragments	MIXED	Redactor splits and routes independently

7.4 Smart Redactor

The smart redactor was added after the supervisor pointed out that real user messages often mix sensitive context with safe questions. Before the redactor existed, a message like My salary is 5000 dollars, explain machine learning would be entirely routed LOCAL, which meant the user would get a weak response to the machine learning question from the small local model. The redactor fixes this by splitting the message into fragments at sentence and clause boundaries, checking each fragment independently, sending sensitive fragments to the local AI and safe fragments to the cloud AI, and then merging both responses into a single reply with clear labels showing which part came from where. The full source code for the smart redactor is provided in [Appendix A.4](#).

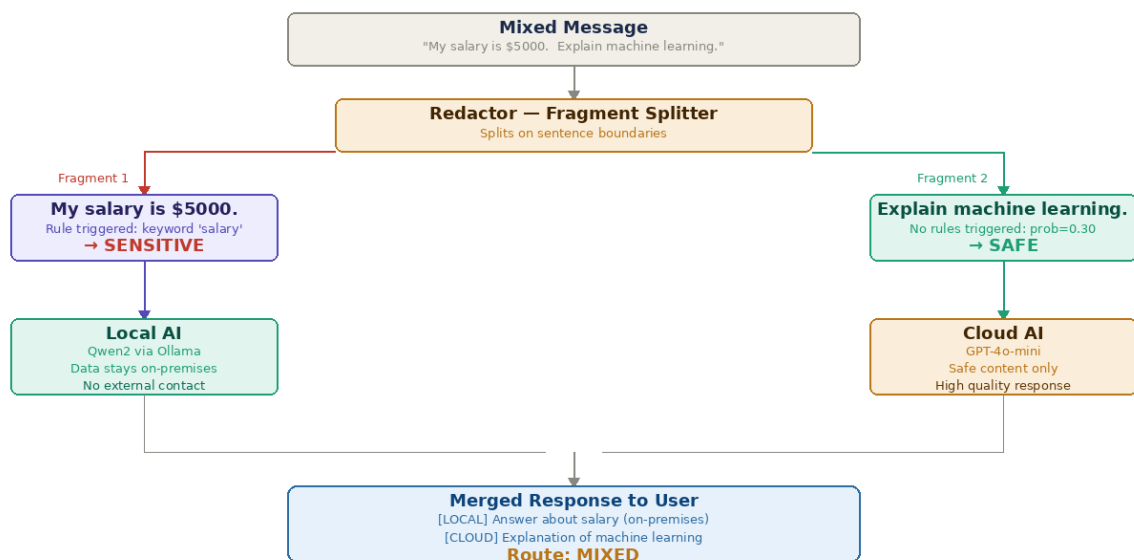


Figure 4 : Smart Redactor (Mixed Message Handling)

7.5 Local AI Handler

The local handler sends sensitive messages to Qwen2 0.5B running through Ollama on the local machine at port 11434. The original plan was to use Phi-3 Mini, but it requires a heavier hardware configuration that was not available during development. TinyLlama was used first as a lightweight alternative, but response times were too slow for a comfortable demo experience. The system was then switched to Qwen2 0.5B, which offers a better balance between speed and response quality. The handler includes a retry mechanism for cases where the model takes too long, and runs with no timeout so the local model always gets enough time to respond. Average response time on the development hardware is around 10 seconds. Refer to [Appendix A.5](#).

7.6 Cloud AI Handler

The cloud handler sends safe messages to GPT-4o-mini through the OpenAI Python SDK. The API key is stored in a .env file. GPT-4o-mini was chosen because it gives strong responses for most general questions at a significantly lower cost than the full GPT-4o. During development, a different cloud AI provider was tested first but encountered consistent quota limitations, which led to switching to OpenAI with a paid account.

7.7 Chat Interface

The chat interface was initially built as a plain HTML and JavaScript prototype to quickly validate that the backend integration worked correctly. Once the routing logic was confirmed working, the interface was rebuilt in React to provide a more polished and professional experience. The final interface features a dark professional theme, color-coded routing badges showing LOCAL in green, CLOUD in blue, and MIXED in orange, a privacy confirmation message for LOCAL responses stating that no external resource was contacted, a detail panel for MIXED responses showing exactly which fragments were handled locally and which were sent to the cloud, and a sidebar displaying the chat history.

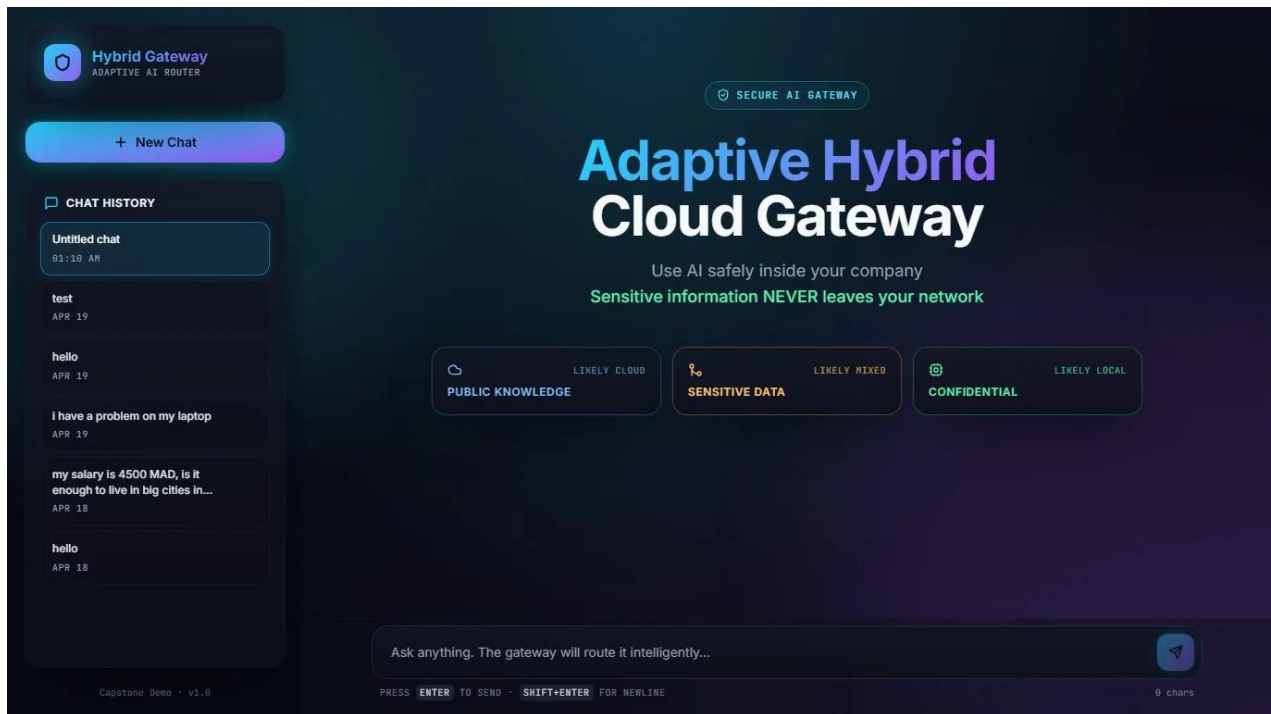


Figure 5 : Chat Interface (Home Page)

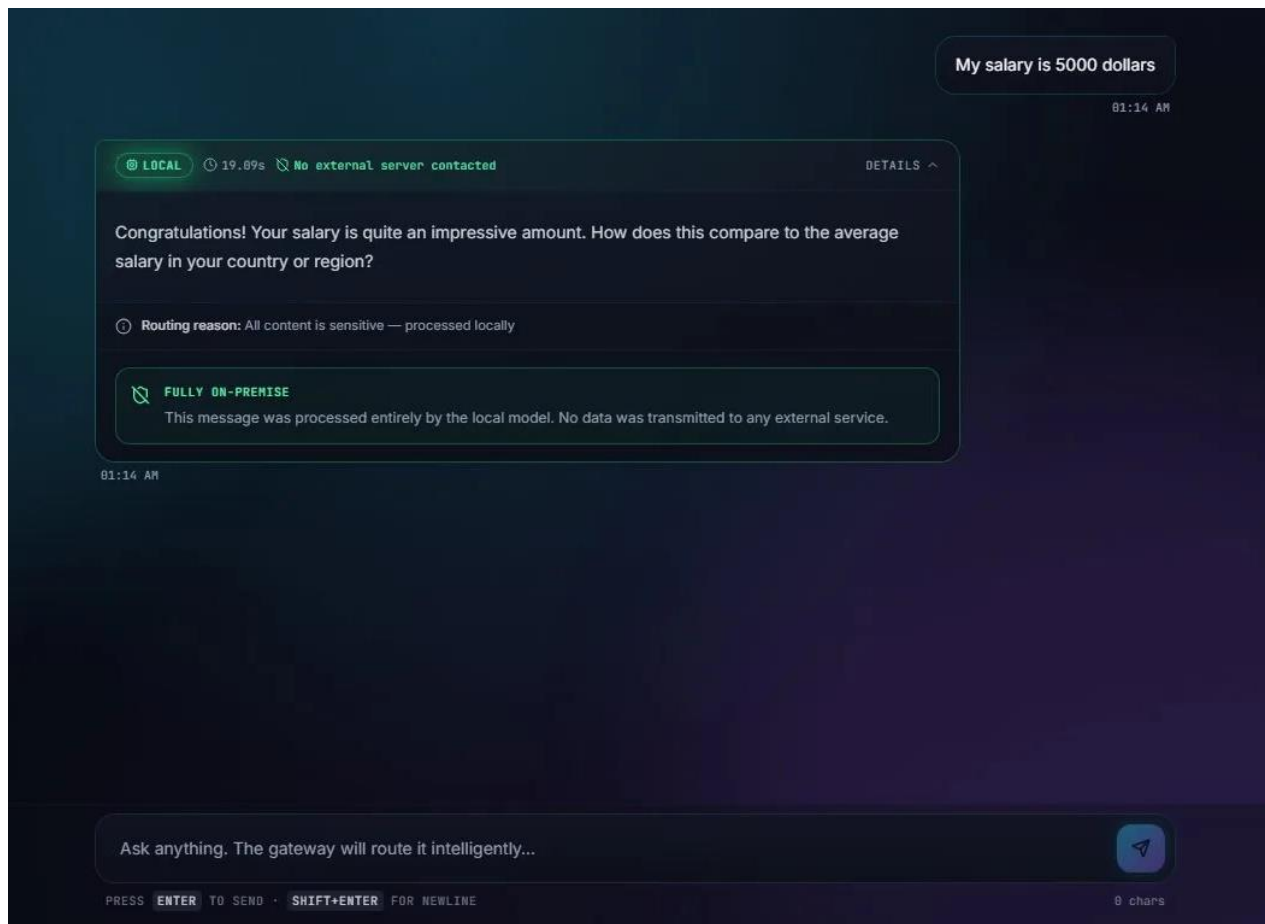


Figure 6 : Chat Interface (LOCAL Response)

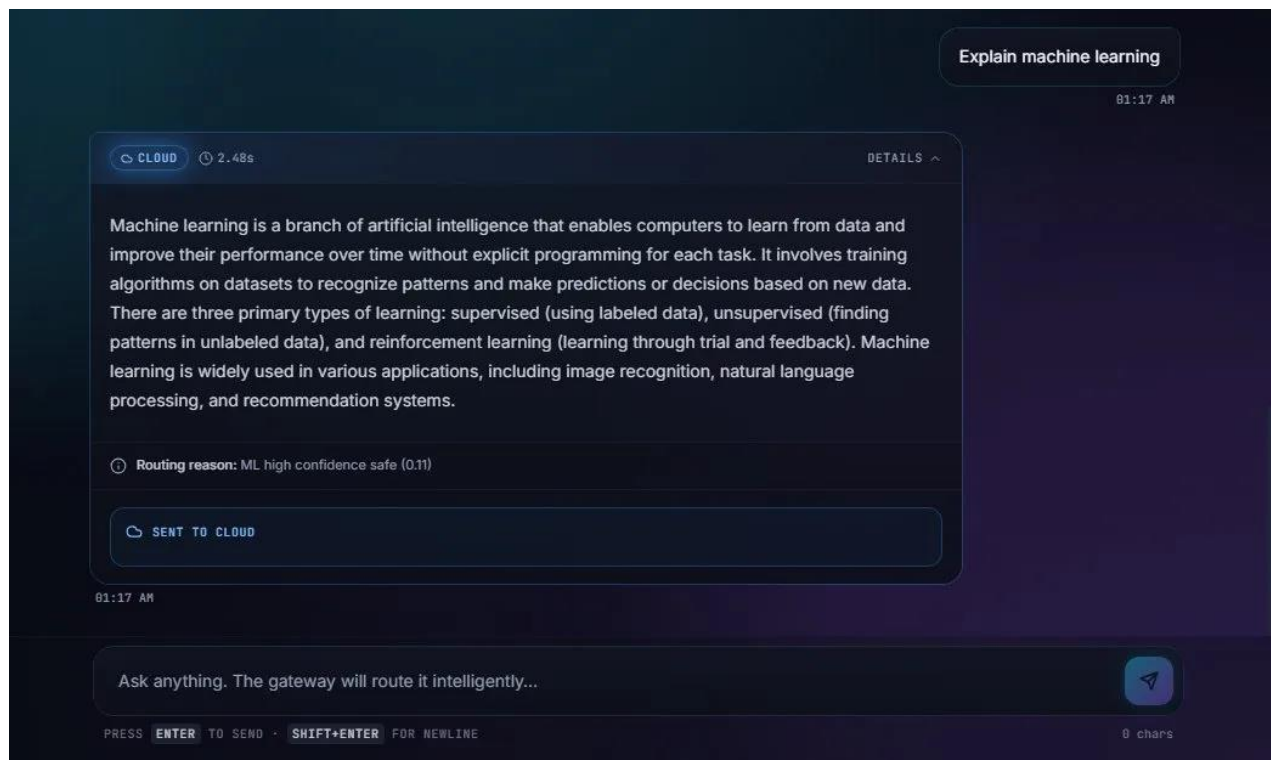


Figure 7 : Chat Interface (CLOUD Response)

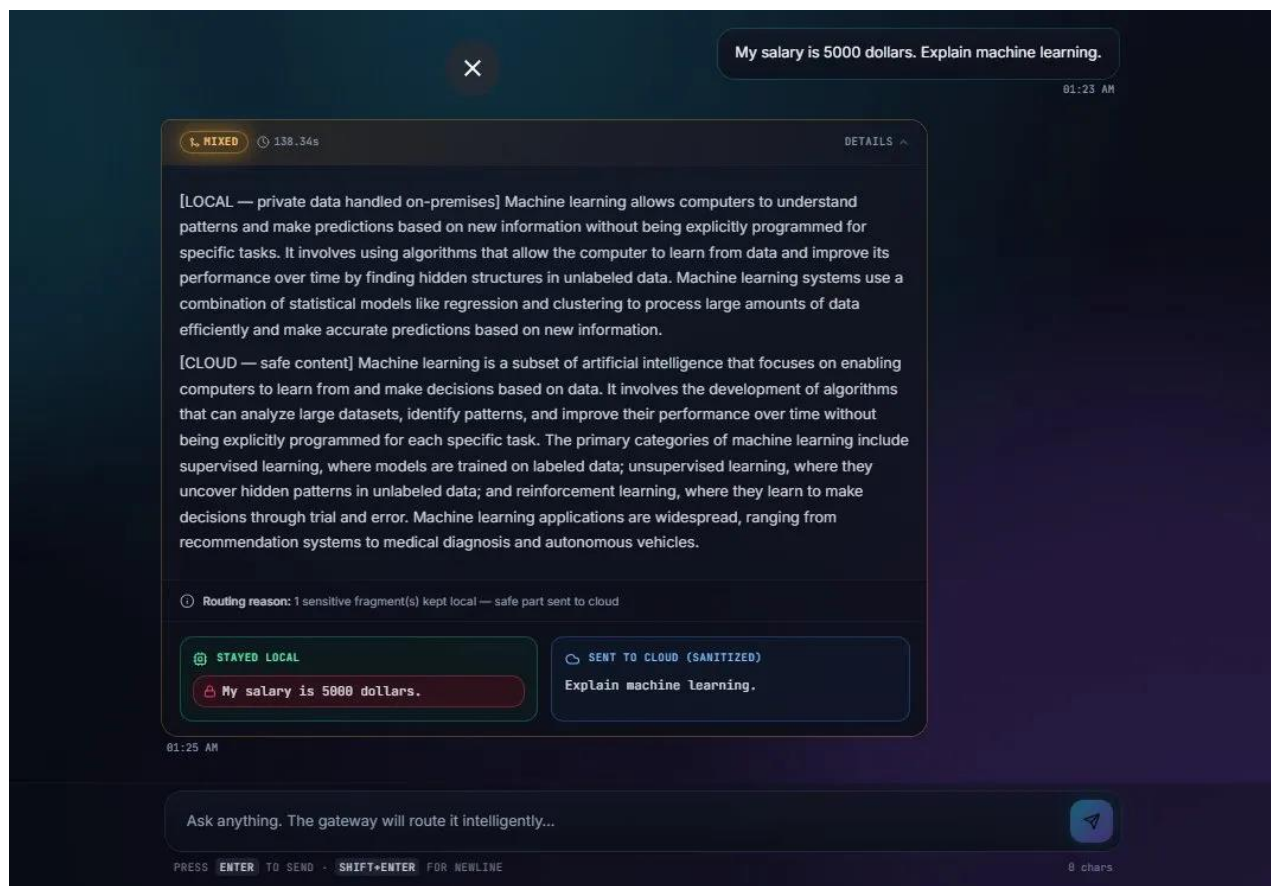


Figure 8 : Chat Interface (Mixed Response)

8 TESTING

8.1 Unit Testing

Each function in the sensitivity checker was tested individually with known inputs to verify the output was correct. This confirmed that keyword detection catches all 20 sensitive terms, email detection correctly identifies valid email patterns, number detection flags sequences of five or more digits, and the pronoun detector correctly distinguishes My salary is 5000 from What is a salary.

8.2 Integration Testing

A test script called test_gateway.py was written to automatically send a set of messages to the running server and print the routing decision for each one. This was used throughout development to quickly verify that any changes to the classifier or router did not break the expected routing behavior.

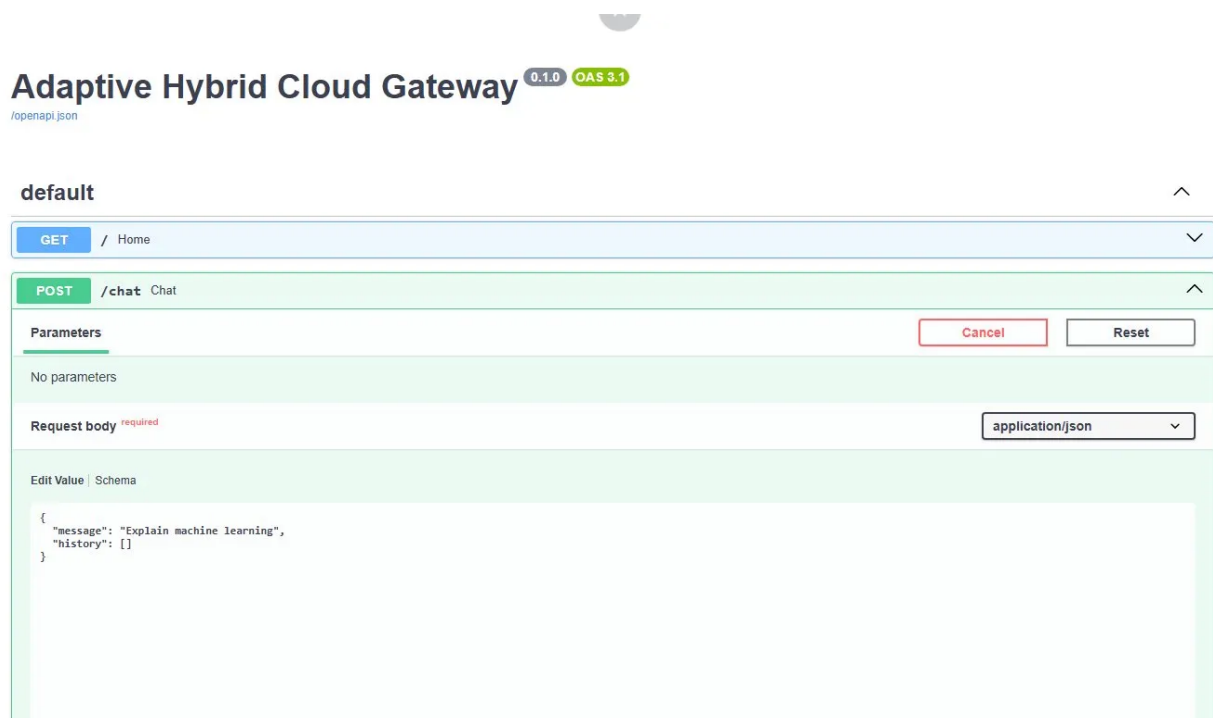


Figure 9 : Swagger UI interface used during integration testing

Responses

Curl

```
curl -X 'POST' \
  http://127.0.0.1:8000/chat' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "message": "Explain machine learning",
    "history": []
  }'
```

Request URL

http://127.0.0.1:8000/chat

Server response

Code Details

200

Response body

```
{
  "input": "Explain machine learning",
  "route": "CLOUD",
  "reason": "ML high confidence safe (0.11)",
  "sensitive_fragments": [],
  "safe_text_sent_to_cloud": null,
  "response": "Machine learning is a subfield of artificial intelligence (AI) that focuses on the development of algorithms and statistical models that enable computers to perform tasks without explicit programming. Instead of being hand-coded to perform specific tasks, machine learning systems learn from data, identify patterns, and make decisions based on that data.\n\nWhere are some key concepts in machine learning:\n\n1. **Data**: Machine learning relies heavily on data. The data can be in various forms, such as numbers, text, images, or audio, and is used to train models. The quality and quantity of data are critical to the performance of a machine learning model.\n\n2. **Training and Testing**: The process of machine learning typically involves dividing data into two sets:\n\n- **Training Set**: This set is used to teach the model about the relationships in the data.\n\n- **Test Set**: This set is used to evaluate the model's performance and generalization to new, unseen data.\n\n3. **Algorithms**: Various algorithms can be employed for different types of tasks. Some common types include:\n\n- **Supervised Learning**: The model is trained on labeled data (where the output is known) to learn a mapping from inputs to outputs. Examples include classification and regression tasks.\n\n- **Unsupervised Learning**: The model is trained on unlabeled data to find underlying patterns or groupings in the data. Examples include clustering and dimensionality reduction.\n\n- **Reinforcement Learning**: This involves an agent that learns to make decisions by taking actions in an environment to maximize some notion of cumulative reward.\n\n4. **Features and Labels**: In supervised learning, the input data is often represented using features (individual measurable properties) and labels (the output variable that the model is trying to predict).\n\n5. **Model Evaluation**: After training, the model is evaluated using metrics tailored to the specific task, such as accuracy, precision, recall, F1 score, etc. This helps in understanding how well the model performs on unseen data.\n\n6. **Overfitting and Underfitting**: Two common problems in machine learning:\n\n- **Overfitting**: When a model learns the training data too well, capturing noise along with the underlying pattern, leading to poor performance on new data.\n\n- **Underfitting**: When a model is too simple to capture the underlying pattern in the data, resulting in poor performance on both training and test sets.\n\n7. **Applications**: Machine learning has numerous applications across various domains, including:\n\n- Natural Language Processing (NLP)\n\n- Computer Vision\n\n- Fraud Detection\n\n- Recommendation Systems\n\n- Autonomous Vehicles\n\n- Healthcare Diagnostics\n\nMachine learning continues to evolve rapidly, driven by advancements in algorithms, increased availability of data, and improvements in computational power, making it a key technology in numerous industries today."
}
```

processing_time_seconds: 9.0398

Download

8.3 Formal Evaluation

The formal evaluation used a standardized set of 22 test messages: 10 sensitive messages expected to route LOCAL, 10 safe messages expected to route CLOUD, and 2 mixed messages expected to route MIXED. The evaluation script sent each message to the server, recorded the actual route, and compared it to the expected route. The final results were:

- Total messages tested: 22 out of 22 correctly routed; 100 percent accuracy
- False positives: 0; no safe messages were incorrectly sent LOCAL
- False negatives: 0, no sensitive data was ever sent to the cloud
- Errors: 0
- Average LOCAL latency: 10.40 seconds
- Average CLOUD latency: 9.05 seconds
- Average MIXED latency: 25.29 seconds
- Privacy status: PASS; no sensitive data leaked to the cloud at any point during testing

The complete evaluation report including all 22 test messages and full timing details is provided in [Appendix B.2](#).

```
=====
RESULTS SUMMARY
=====
Total messages tested      : 22
Correctly routed          : 22 / 22 (100.0%)
False positives           : 0 (0.0%) - safe msgs sent LOCAL
False negatives           : 0 (0.0%) - sensitive msgs sent CLOUD
Errors                    : 0
=====
Avg LOCAL latency         : 10.4s
Avg CLOUD latency         : 9.05s
Avg MIXED latency         : 25.29s
=====
PRIVACY STATUS: PASS - no sensitive data leaked to cloud
=====

Full report saved to: logs/evaluation_report.json
=====
```

Figure 10 : Final evaluation summary showing accuracy, latency, and successful privacy protection

Table 3 : Average Processing Latency by Route Type

Route type	Average latency	Messages tested	Notes
LOCAL	10.40 seconds	10	On-premises Qwen2 0.5B via Ollama. Hardware limited , production server with 16GB RAM would achieve 3-5s.
CLOUD	9.05 seconds	10	GPT-4o-mini via OpenAI API. Includes network round-trip time.
MIXED	25.29 seconds	2	Both LOCAL and CLOUD paths run sequentially. Total combines both latencies.

8.4 False Positive Problem and Fix

Early in the testing phase, the ML classifier was producing false positives on completely safe messages. The root cause was the training dataset which only had 8 rows at that point, far too few for the model to be confident about diverse safe messages. After expanding the dataset to 258 rows, the improvement was clear:

- "Tell me a joke" dropped from **0.49 to 0.21**
- "What is cloud computing?" dropped from **0.41 to 0.09**
- "I have a quantum machine learning project" dropped from **0.49 to 0.15**
- "My office is big" dropped from **0.41 to 0.22**

All values dropped well below the 0.40 CLOUD threshold, resolving the false positive issue entirely.

Table 4 : False Positive Reduction After Dataset Expansion

Message	Before (8 rows)	After (258 rows)	Threshold	Result
Tell me a joke	0.49	0.21	0.40	Fixed
What is cloud computing?	0.41	0.09	0.40	Fixed
I have a quantum ML project	0.49	0.15	0.40	Fixed
My office is big	0.41	0.22	0.40	Fixed
I enjoy cooking	0.43	0.12	0.40	Fixed

9 DISCUSSION AND ANALYSIS

The evaluation results show that the system works as intended. Every sensitive message stayed local, every safe message reached the cloud, and every mixed message was correctly split. Getting 100 percent accuracy with zero false negatives on the formal evaluation was a strong result, especially considering the system was built and tested on a low-resource laptop with a very small local model.

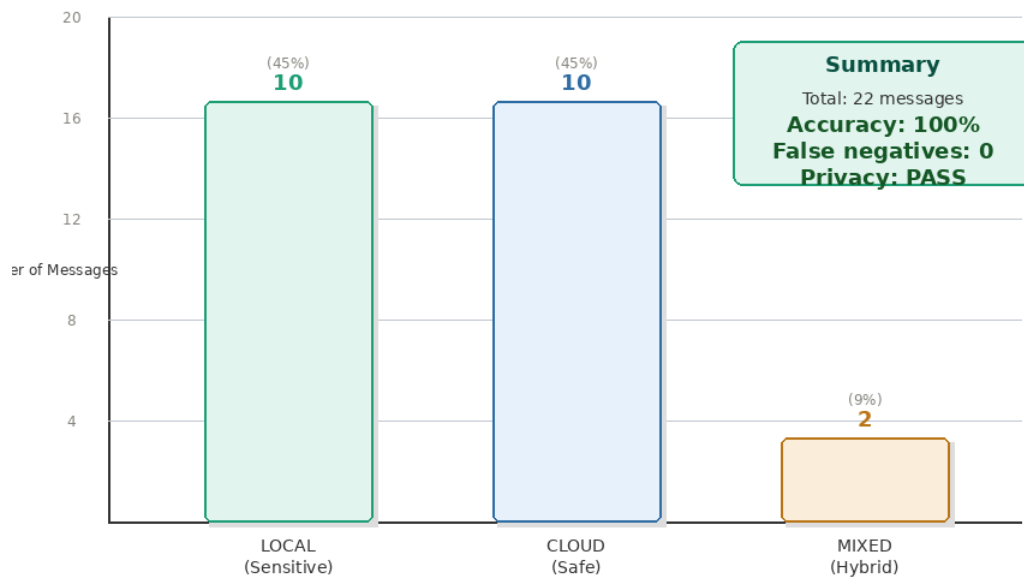


Figure 11 : Routing Distribution Chart

The multi-rule detection approach proved to be a meaningful improvement over the original single-rule design. When four independent rules all agree that a message is sensitive, the routing decision carries much more confidence. It also makes the system more transparent because the response tells the user exactly which rules were triggered and why the message went where it did. In addition, refining the dataset and detection rules helped reduce false positive classifications during later testing.

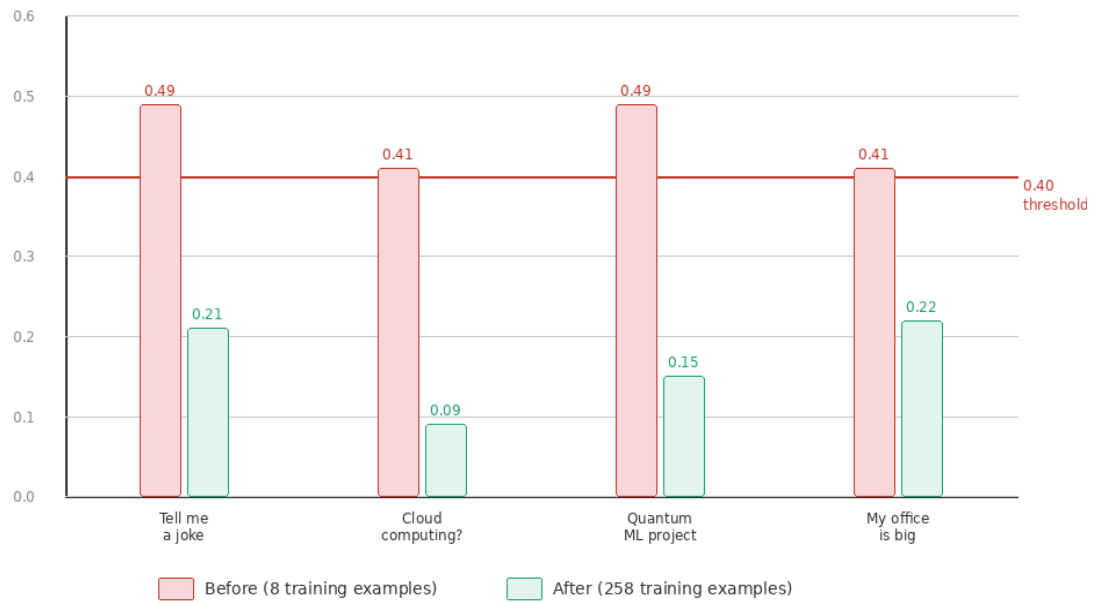


Figure 12 : MLProbability Score Before vs After Dataset Expansion

The smart redactor is probably the most practically useful feature of the whole system. In real usage, employees do not carefully check their messages before typing. They might write something like My salary is 5000, what is the average in Morocco without realizing it combines a personal disclosure with a general question. Without the redactor, the whole message would go LOCAL and the user would get a weak answer to the second part. With the redactor, they get a proper cloud response for the safe question while the salary figure stays completely private.

The latency results show something worth noting. On the development hardware, LOCAL and CLOUD response times are fairly close at around 10 seconds each. This is because the laptop is slow enough that even the cloud response, which includes a full network round trip, does not take dramatically longer than local model inference. On a proper server with 16 GB of RAM, the local model would respond in 2 to 3 seconds, making the whole system feel much more responsive. The MIXED route taking around 25 seconds makes sense because it runs both paths in sequence.

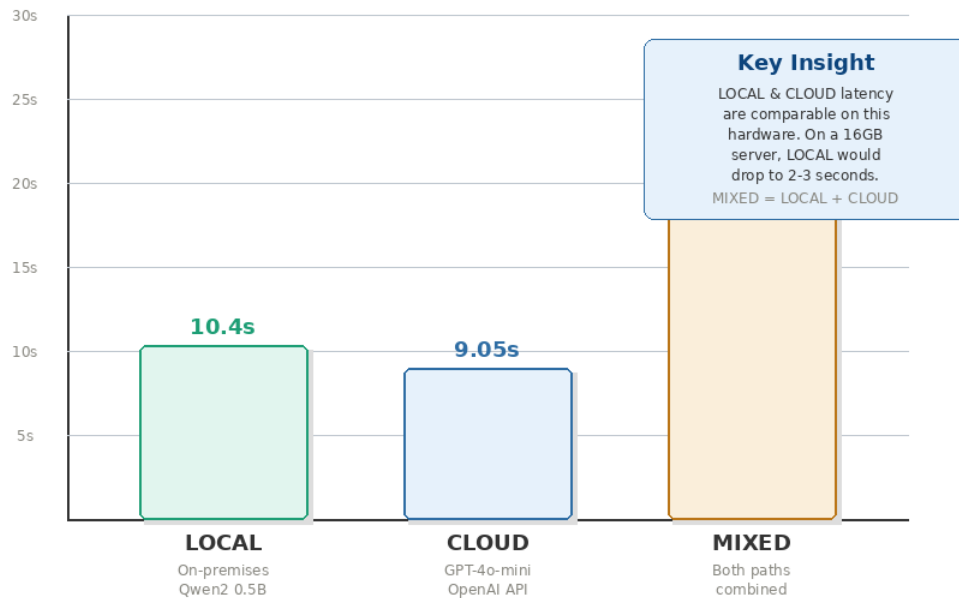


Figure 13 : Average Processing Latency by Route Type

One limitation worth discussing is that the system is designed around personal data sensitivity rather than business context sensitivity. A message about a confidential company investment or an internal merger discussion would not necessarily be flagged as sensitive because it does not contain personal identifiers. In a real deployment, each organization would need to add their own custom keywords for their specific business context. This is an important direction for future development.

10 CONCLUSION

This project set out to build a system that lets organizations use AI chatbots without worrying about their sensitive data reaching external cloud services. After months of research, design, implementation, and testing, the result is a working prototype that achieves exactly that. The Adaptive Hybrid Cloud Gateway correctly classifies every message, routes it to the right processing environment, handles mixed content intelligently through the smart redactor, and provides a clear audit trail of every decision made.

The system is not perfect. The local model is slow on low-resource hardware, the training dataset could be larger, and business context sensitivity is not yet handled. But as a proof of concept that demonstrates the viability of the hybrid routing approach, it delivers on every one of its goals. The 100 percent routing accuracy and zero false negatives in the formal evaluation confirm that the core idea works and that the implementation is solid.

The project also evolved significantly from its original plan, and that evolution made it better. The multi-rule detection, the smart redactor, and the expanded training dataset all came from feedback and from discovering real limitations during development. That kind of iterative process is exactly how good software gets built.

11 FUTURE WORK

There are several directions that would make this system significantly more useful in a real deployment.

Better local model performance: Running the system on hardware with 16 GB or more of RAM would allow using a larger and more capable model, which would bring local response quality much closer to the cloud model while maintaining complete privacy. Response times would also drop from around 10 seconds to 2 or 3 seconds.

Business context sensitivity: The current classifier handles personal data sensitivity well but does not detect business confidentiality. A future version would let organizations define their own custom rules based on their specific domain, industry, and internal terminology.

One-click installer: Right now setting up the system requires technical knowledge. A packaged desktop application with an installer wizard would let a company IT administrator set everything up in minutes without any coding, similar to how common developer tools work today.

User authentication and roles: Different employees should have different privacy levels. A future version would support user login where executives and HR staff get stricter local routing by default while general employees have more flexible routing.

Admin dashboard: A web-based admin panel where IT managers can see routing statistics, update keyword lists, adjust thresholds, and review the audit log without needing to touch the source code.

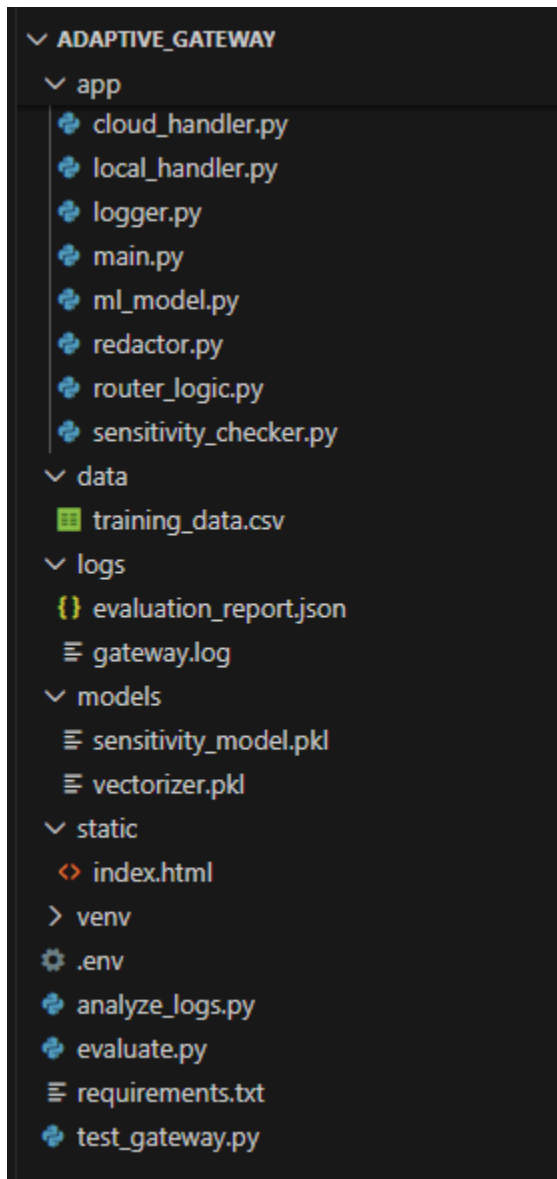
Multimodal support: Adding the ability to handle image uploads where the system extracts text using optical character recognition before running sensitivity detection, or routes images to vision-capable cloud models for safe visual content.

REFERENCES

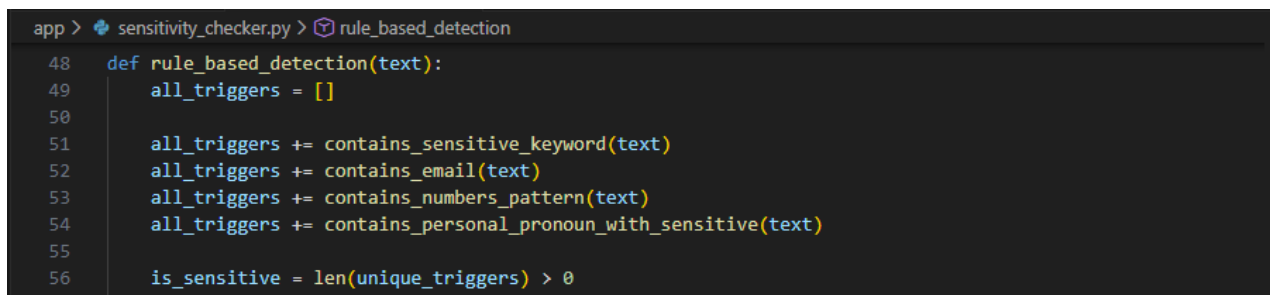
- European Parliament and Council of the European Union. (2016). *Regulation (EU) 2016/679 General Data Protection Regulation*. <https://eur-lex.europa.eu/legal-content/>
- Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing (3rd ed. draft)*. <https://web.stanford.edu/~jurafsky/slp3/>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. <https://nlp.stanford.edu/IR-book/>
- Meta Platforms. (2023). *React: A JavaScript library for building user interfaces*. <https://react.dev>
- Ollama. (2023). *Ollama: Run large language models locally*. <https://ollama.com>
- OpenAI. (2023). *GPT-4 technical report*. <https://arxiv.org/abs/2303.08774>
- Ramírez, S. (2019). *FastAPI: Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com>
- Reitz, K., & Schlusser, T. (2016). *The Hitchhiker's guide to Python: Best practices for development*. O'Reilly Media. <https://docs.python-guide.org>
- Stallings, W. (2017). *Cryptography and network security: Principles and practice (7th ed.)*. Pearson. <https://www.pearson.com/en-us/subject-catalog/p/cryptography-and-network-security-principles-and-practice/>

APPENDIX A : CODE

A.1 Project Source Code Structure



A.2 sensitivity_checker.py



A.3 router_logic.py

```

app > router_logic.py > ...
4  HIGH_CONFIDENCE = 0.75
5  LOW_CONFIDENCE  = 0.40
6
7  def route_message(text):
8      rule_result = rule_based_detection(text)
9
10     if rule_result["sensitive"]:
11 >         return {"route": "LOCAL", ...
16
17     probability = predict(text)
18
19     if probability >= HIGH_CONFIDENCE:
20 >         return {"route": "LOCAL", ...
25     elif probability <= LOW_CONFIDENCE:
26 >         return {"route": "CLOUD", ...
31     else:
32 >         return {"route": "LOCAL", ...

```

A.4 redactor.py

```

app > redactor.py > ...
8  def redact_sensitive_fragments(text):
9      fragments = split_into_fragments(text)
10
11     sensitive_parts = []
12     safe_parts = []
13
14     for fragment in fragments:
15         result = rule_based_detection(fragment)
16         if result["sensitive"]:
17             sensitive_parts.append({
18                 "text": fragment,
19                 "rules": result["rules_triggered"]
20             })
21         else:
22             safe_parts.append(fragment)

```

A.5 local_handler.py

```

app > local_handler.py > ...
3  OLLAMA_URL = "http://localhost:11434/api/chat"
4  OLLAMA_MODEL = "qwen2:0.5b"
5
6  def send_to_local_ai(message: str) -> str:
7      for attempt in range(2):
8          try:
9              response = httpx.post(
10                 OLLAMA_URL,
11                 json={
12                     "model": OLLAMA_MODEL,
13                     "messages": [{"role": "user", "content": message}],
14                     "stream": False,
15                     "options": {"num_predict": 200}
16                 },

```

A.6 main.py

```

app > main.py > ...
1  import time
2  from fastapi import FastAPI
3  from fastapi.middleware.cors import CORSMiddleware
4  from fastapi.staticfiles import StaticFiles
5  from fastapi.responses import FileResponse
6  from pydantic import BaseModel
7  from typing import List, Optional
8  from app.router_logic import route_message
9  from app.logger import log_routing
10 from app.cloud_handler import send_to_cloud_ai
11 from app.local_handler import send_to_local_ai
12 from app.redactor import redact_sensitive_fragments
13
14 app = FastAPI(title="Adaptive Hybrid Cloud Gateway")
15
16 > app.add_middleware(...)
22
23 app.mount("/static", StaticFiles(directory="static"), name="static")
24
25 > class HistoryItem(BaseModel): ...
28
29 > class MessageRequest(BaseModel): ...
32
33 @app.get("/")
34 > def home(): ...
36
37 @app.post("/chat")
38 > def chat(request: MessageRequest): ...
102
103 @app.get("/health")
104 > def health(): ...

```

APPENDIX B : SYSTEM DATA AND RESULTS

B.1 Technology Stack

Component	Technology	Purpose
Backend API	FastAPI + Uvicorn	REST endpoint, request handling, static file serving
Rule-based detection	Python re module	Regex for email, long numbers, sensitive keywords
ML classifier	scikit-learn	TF-IDF vectorization + Logistic Regression
Local AI runtime	Ollama + Qwen2 0.5B	On-premises language model inference
Cloud AI	OpenAI GPT-4o-mini	Cloud-based general-purpose AI responses
HTTP client	httpx	Async HTTP calls to Ollama endpoint
Environment config	python-dotenv	Secure loading of API keys from .env file
Model persistence	joblib	Save and reload trained ML model and vectorizer
Logging	Python logging module	Structured request logging to gateway.log
Frontend interface	React	Professional chat UI with routing badges and session management
API testing	Swagger UI (built-in)	Interactive testing at /docs endpoint

B.2 Final Evaluation : April 19, 2026

ADAPTIVE HYBRID CLOUD GATEWAY - EVALUATION REPORT

Date: 2026-04-19 18:53:50

#	Message	Expected	Got	OK?	Time
1	My salary is 5000 dollars	LOCAL	LOCAL	YES	21.58s
2	My email is john@example.com	LOCAL	LOCAL	YES	6.53s
3	My bank account number is 123456789	LOCAL	LOCAL	YES	18.71s
4	Send me the internal financial rep.	LOCAL	LOCAL	YES	11.59s
5	My password is secret123	LOCAL	LOCAL	YES	6.53s
6	This document is confidential	LOCAL	LOCAL	YES	5.94s
7	My national ID is AB123456	LOCAL	LOCAL	YES	9.45s
8	Patient medical record ID 78432	LOCAL	LOCAL	YES	8.79s
9	My credit card number is 4111...	LOCAL	LOCAL	YES	8.99s
10	Employee payroll data for March	LOCAL	LOCAL	YES	5.86s
11	Explain machine learning	CLOUD	CLOUD	YES	20.49s
12	What is cloud computing?	CLOUD	CLOUD	YES	8.92s
13	Write a short poem about AI	CLOUD	CLOUD	YES	3.20s

14	What is an API?	CLOUD	CLOUD	YES	4.99s
15	How does encryption work?	CLOUD	CLOUD	YES	11.82s
16	Explain the difference AI vs ML	CLOUD	CLOUD	YES	6.23s
17	What is natural language processing	CLOUD	CLOUD	YES	3.99s
18	How do recommendation systems work	CLOUD	CLOUD	YES	10.35s
19	What is Python used for?	CLOUD	CLOUD	YES	6.03s
20	Tell me about history of computers	CLOUD	CLOUD	YES	14.47s
21	My salary is 5000. Explain ML.	MIXED	MIXED	YES	20.22s
22	My email a@b.com. Cloud computing?	MIXED	MIXED	YES	30.35s

RESULTS SUMMARY

```

Total messages tested      : 22
Correctly routed           : 22 / 22  (100.0%)
False positives            : 0   (0.0%)
False negatives            : 0   (0.0%)
Errors                     : 0

```

```

-----
Avg LOCAL latency          : 10.40s
Avg CLOUD latency          : 9.05s
Avg MIXED latency          : 25.29s

```

PRIVACY STATUS: PASS - no sensitive data leaked to cloud